



清华大学计算机科学与技术系本科专业主修课

计算机系统结构

第四章 向量处理器

(课程复习)

■ 发挥向量处理机的计算优势：

关键点一：从标量表示到向量数据的表示方式：

1. 等间距向量表示法
2. 带位移量的向量表示法
3. 稀疏向量表示法

关键点二：在向量处理机上实现有3种处理方式：

1. **横向处理方式**，又称水平处理方式，横向加工方式等，向量计算是按行的方式从左至右横向地进行
2. **纵向处理方式**，又称垂直处理方式，纵向加工方式等，向量计算是按列的方式自上而下纵向地进行
3. **纵横处理方式**，又称分组处理方式，纵横加工方式等，横向处理和纵向处理相结合的方式

向量处理机的结构

第 3 页

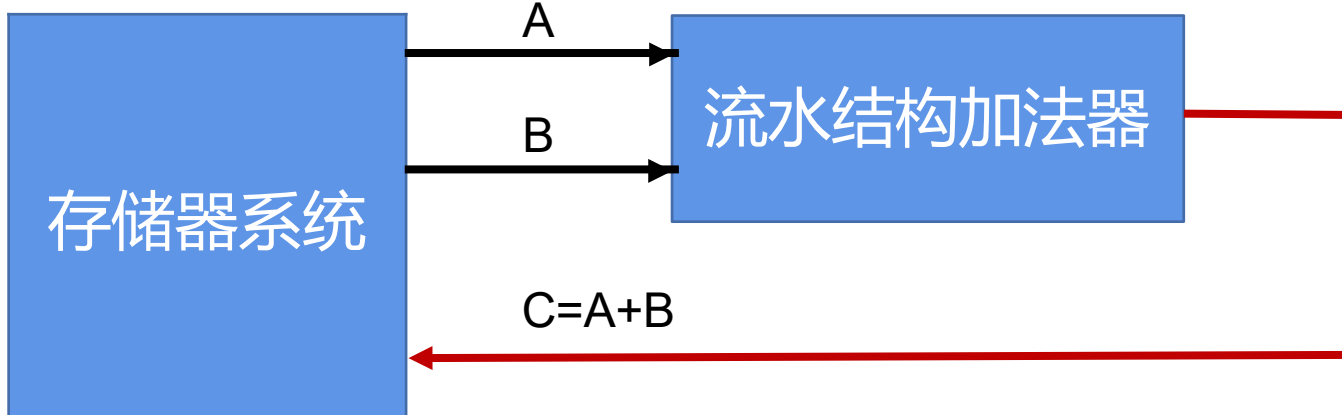
向量处理机的基本思想是把两个向量的对应分量进行运算，产生一个结果向量。

例：如果A、B、C都是向量，各有N个元素，则一台向量处理机能够完成如下运算：

$$C_i = A_i + B_i$$

$0 \leq i \leq N-1$ ，其中A、B、C用分量形式表示

采用流水线运算部件实现上述运算：



如果要求取操作数、运算、写结果回存储器在一个时钟周期内完成，则要求存储系统能在一个时钟周期内读出两个操作数和写回一个运行结果。



存储器系统的带宽至少应3倍于一般的存储器系统

- 向量处理机的结构由所采用的向量处理方式决定，有两种典型结构
 - **存储器-存储器型结构**：纵向处理方式采用，利用几个独立的存储器模块来支持向对独立的数据并发访问
 - ✓ 多个独立得到存储模块并行工作
 - ✓ 处理机结构简单，对存储系统的访问速度要求很高
 - **寄存器-寄存器型结构**：分组处理方式采用，构造一个具有所要求带块的高速中间寄存器，实现高速中间寄存器与主寄存器之间的快速数据交换
 - ✓ 运算通过向量寄存器实现，需要大量高速寄存器
 - ✓ 对存储系统访问速度要求低

“存储器-存储器”结构

第 5 页

解决访问冲突的方法之一：把存储体的个数 n 选为质数，且 $n \geq$ 向量长度，变址位移量就必然与 n 互质，一维数组的访问冲突自然也就不存在了。

- 例如美国研制的BSP巨型计算机的存储体个数为17，我国研制的银河巨型计算机的存储体个数为37。

$m \times L$ 不能整除 n



地址 $A_i + m \times L$ ($L=1 \sim 16$) 访问的存储器与地址 A_i 访问的存储器是一致的概率是0

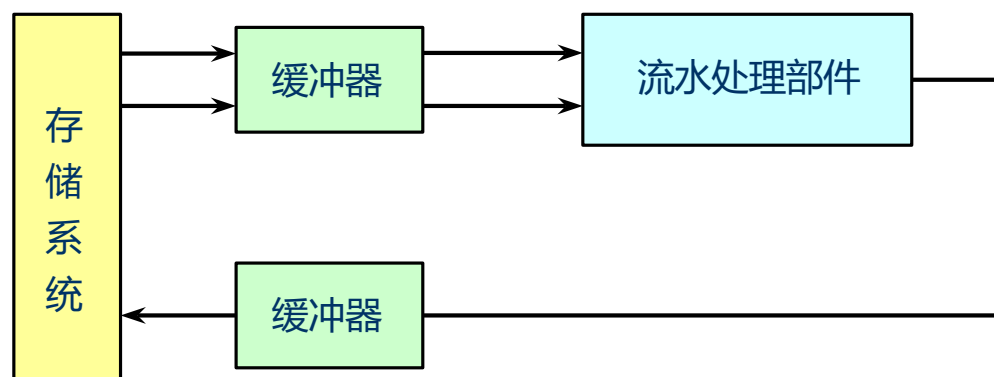
体内地址	0号体	1号体	2号体	3号体	4号体
0	A0	A1	A2	A3	A4
1	A5	A6	A7	A8	A9
2	A10	A11	A12	A13	A14
3	A15	A16	A17	A18	A19

存在问题：虽然采用质数个存储体能够缓解访问存储器的冲突，但是数据经过多次运算之后，在存储体中的分布必然发生改变，无法克服访问冲突问题。

“存储器-存储器”结构

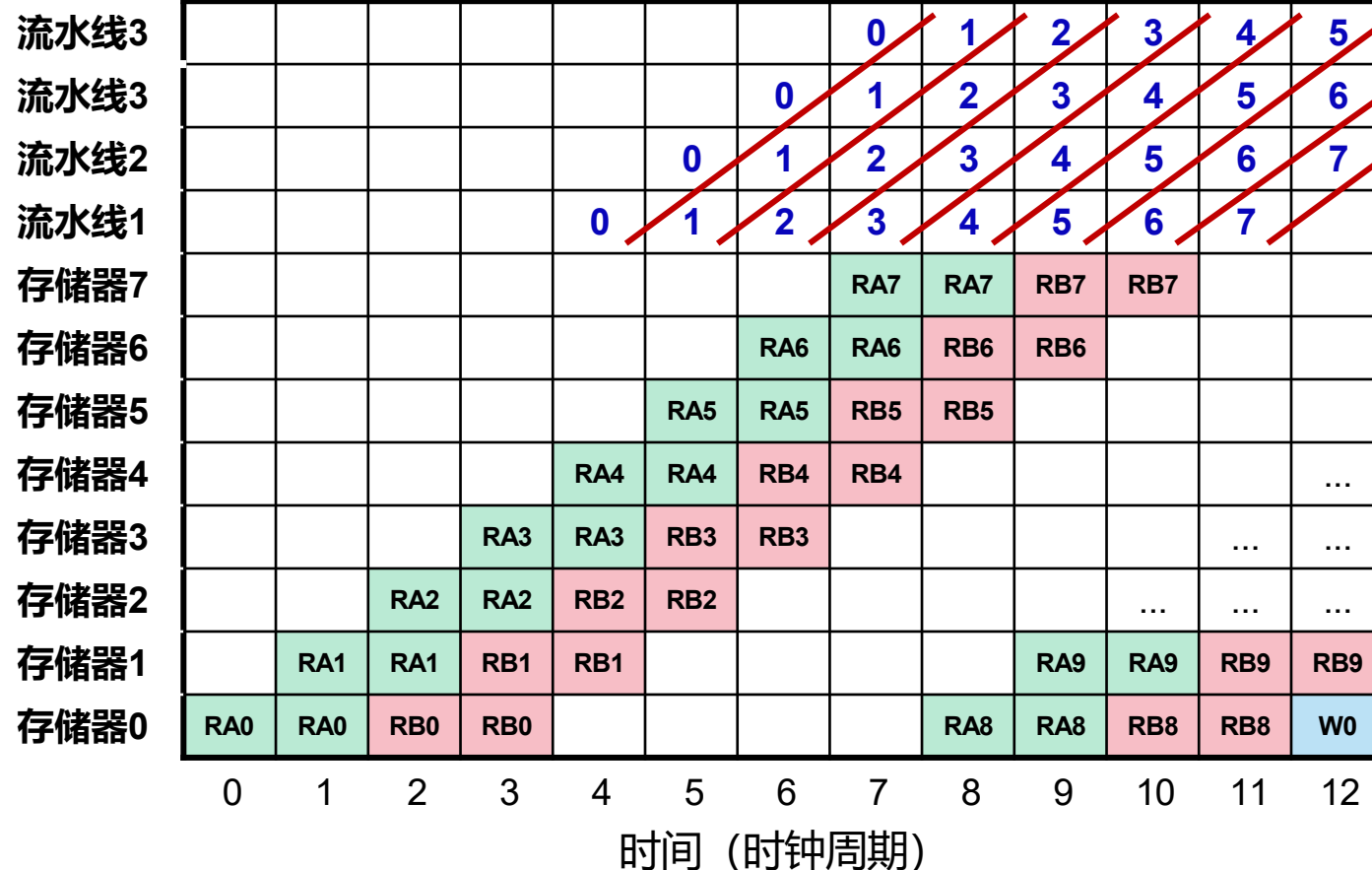
第 6 页

解决访问冲突的方法之二：在运算流水线的输入端和输出端增加操作数**可变缓冲器**和写结果**可变缓冲器**，以便消除征用存储器的现象



方法：向量A的输入缓冲器延迟**2**个时间周期；输出缓冲器延迟**4**个时间周期

结果：向量A和B的对应元素同时到达流水结构加法器；输出缓冲器将每个输出值延迟**4**个时间周期再送入存储器系统，第1个结果在时钟周期**12**时写入主存储器。



“寄存器-寄存器”结构

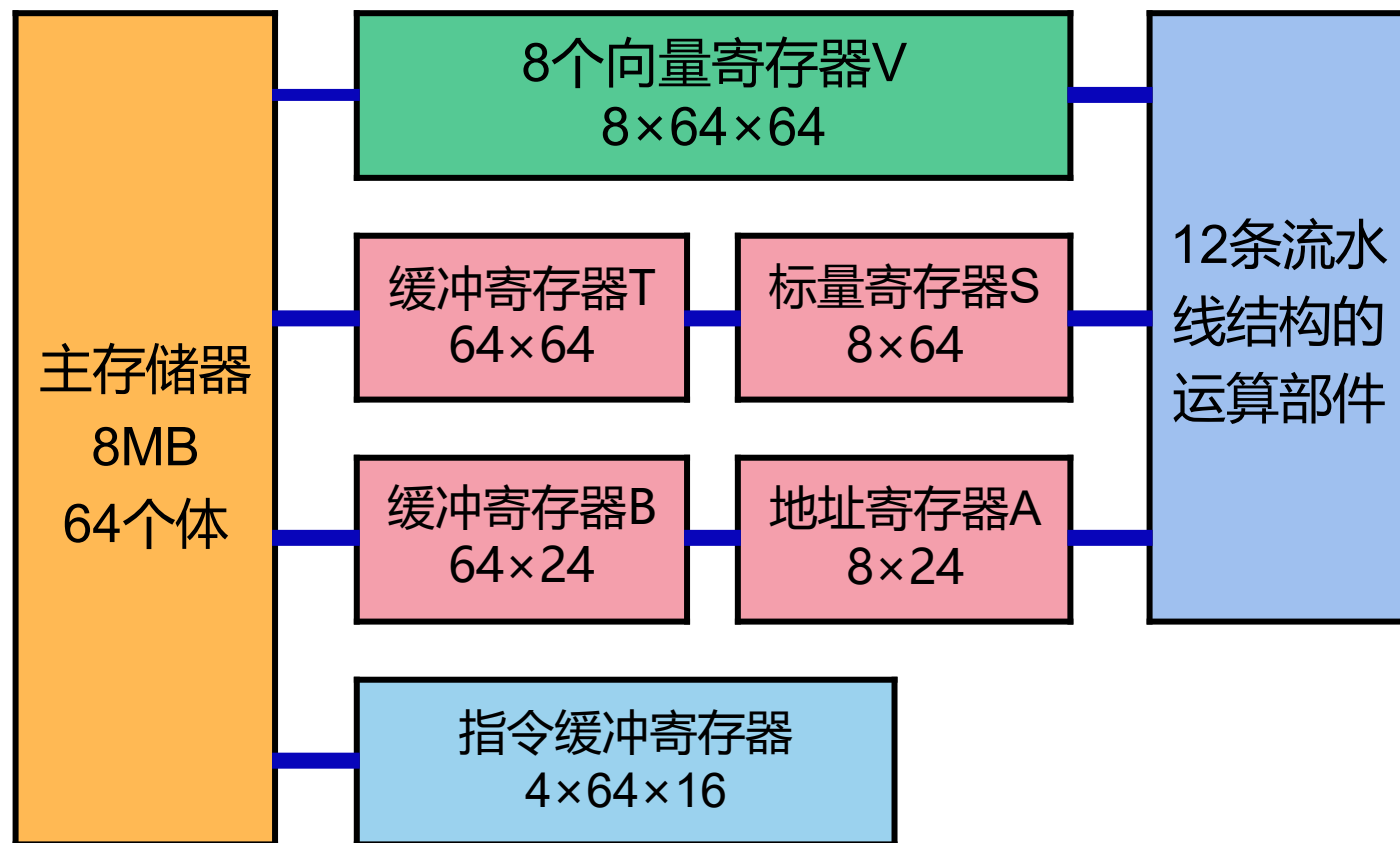
第 7 页

■典型的寄存器 - 寄存器结构的向量处理机：美国的CRAY-1、我国的YH-1巨型机等

■以CRAY-1机为例

- 美国CRAY公司
- 1976年
- 每秒1亿次浮点运算
- 时钟周期：12.5ns

与存储器-存储器结构的STAR-100比较，
运算速度3倍多（时钟周期为40:12.5），
主存储器流量低2.5倍。

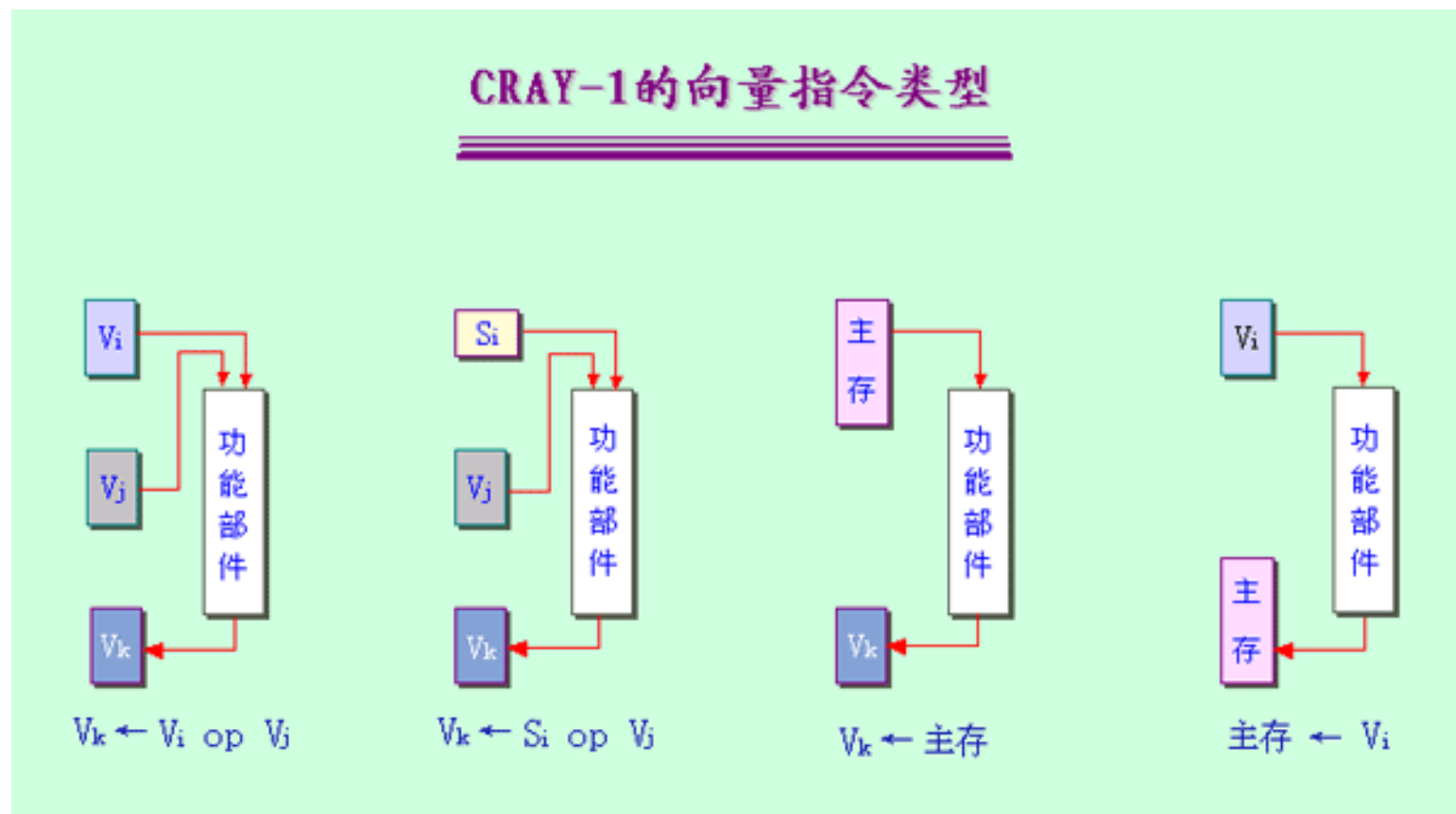


“寄存器-寄存器”结构

第 8 页

CRAY-1 向量指令类型

- ① $V_k \leftarrow V_i \text{ op } V_j$
- ② $V_k \leftarrow S_i \text{ op } V_j$
- ③ $V_k \leftarrow \text{主存}$
- ④ $\text{主存} \leftarrow V_i$



CRAY-1 向量处理冲突

■ 主要类型：V_i冲突和功能部件冲突

- V_i冲突：并行工作的各向量指令的源向量或结果向量使用了相同的V_i，如源向量相同

$$V_3 \leftarrow \mathbf{V_1} + V_2$$

$$V_5 \leftarrow V_4 \wedge \mathbf{V1}$$

- 功能部件冲突：并行工作的各向量指令要使用同一个功能部件，如都需使用乘法功能部件

$$V_3 \leftarrow V_1 \times V_2$$

$$V_5 \leftarrow V_4 \times V_6$$

提高向量处理机性能的方法

- 设置多个功能部件，使它们并行工作；
- 采用链接技术，加快一串向量指令的执行；
- 采用分段开采技术，加快循环的处理；
- 采用多处理机系统，进一步提高性能。

2. 向量流水线冲突分析

■设计目标：保证处理器一起处于忙状态，导致无法满负荷工作的原因是向量流水线冲突。

■向量流水线冲突分析：两条向量指令占用功能流水线和向量寄存器的4种情况：

① 指令不相关

例如： $V0 \leftarrow V1 + V2$

$V6 \leftarrow V4 * V5$

这两条指令分别使用各自所需的流水线和向量寄存器，可以并行执行。

② 功能部件冲突

例如： $V3 \leftarrow V1 + V2$

$V6 \leftarrow V4 + V5$

这两条指令都要使用加法流水线，发生了功能部件冲突（但向量寄存器不冲突）。当第一条指令流出时，占用加法流水线。第二条指令要等加法流水线变成空闲后，才能流出。

③ 源寄存器冲突

例如： $V3 \leftarrow V1 + V2$

$V6 \leftarrow V1 * V4$

这两条向量指令的源向量之一都取自V1。由于两者的首元素下标可能不同，向量长度也可能不同，所以难以由V1同时提供两条指令所需要的源向量。这两条向量指令不能同时执行。只有等第一条向量指令执行完、释放V1之后，第二条向量指令才能开始执行。

④ 结果寄存器冲突——两条向量指令使用了相同的结果向量寄存器。

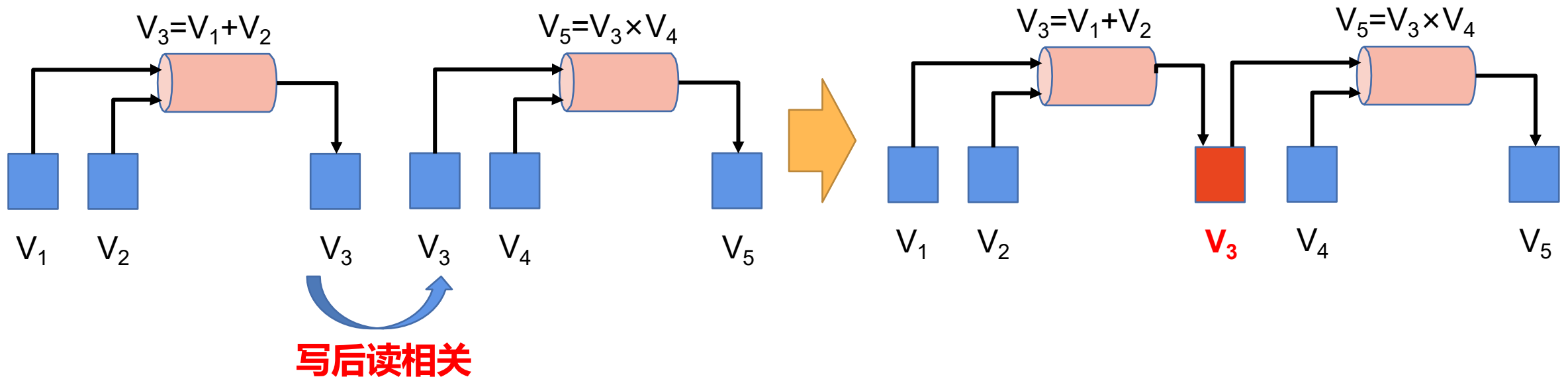
例如： $V4 \leftarrow V1 + V2$

$V4 \leftarrow V3 * V5$

这两条指令都要访问目的寄存器V4。由于第一条指令在先，所以它先占用V4直到运算完成，然后再流出后一条指令。

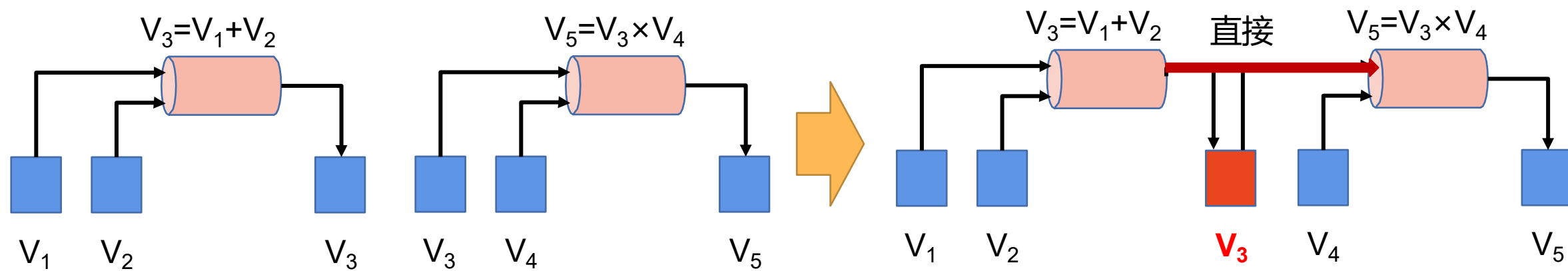
向量流水线链接技术：具有先写后读相关的两条指令，在不出现功能部件冲突和源向量冲突的情况下，可以把功能部件链接起来进行流水处理，以达到加快执行的目的。

■前一条指令的结果不必送回存储器，①直接作为后一条指令的操作数



向量流水线链接技术：

- 前一条指令的结果不必送回存储器，②甚至可在前一条指令完成之前就使用其结果。



结果寄存器可能成为后继指令的操作数寄存器，在CRAY-1中这种技术被称为两条流水线的链接 (chaining)

衡量向量处理机性能的主要参数：

- 向量指令的处理时间
- 向量长度为无穷大时的向量处理机的最大性能
- 半性能向量长度
- 向量长度临界值

1. Cray-1向量处理的一个显著特点是：只要不出现向量寄存器冲突和功能部件冲突，各V之间和各功能部件之间都能并行工作；
2. 把能在同一个时钟周期内一起开始执行的几条向量指令称为一个编队；
3. 在向量流水处理机上，向量指令序列中的一个编队内的指令可以同时执行，编队执行时间为编队内所有的向量指令执行时间的最大值。



清华大学计算机科学与技术系本科专业主修课

计算机系统结构

第5章 指令级并行及其开发—硬件方法

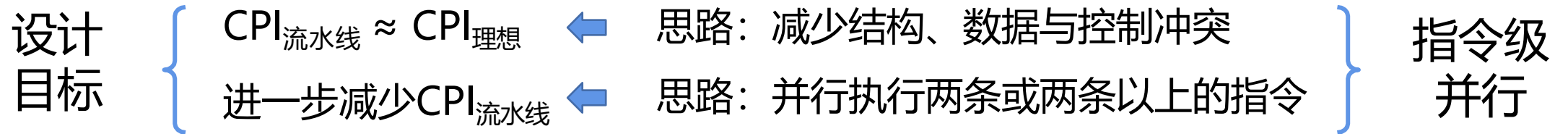
(课程复习)

1. 流水线处理机的实际CPI分析

- 理想流水线的CPI加上各类停顿的时钟周期数：

$$CPI_{\text{流水线}} = CPI_{\text{理想}} + \text{停顿}_{\text{结构冲突}} + \text{停顿}_{\text{数据冲突}} + \text{停顿}_{\text{控制冲突}}$$

- 理想CPI是衡量流水线最高性能的一个指标。



- 指令级并行指机器语言一级，如存储器访问指令、整型指令、浮点指令之间的并行性等。
- 指令级并行由处理器硬件和编译程序自动识别和利用，**不需要程序员对顺序程序作任何修改。**

2. 指令级并行设计对象

- **基本程序块**：一串连续的代码（同时执行相邻或相近的多条指令），除了入口和出口以外没有其他的分支指令和转入点
- **分支指令和转入点**：程序平均每4 ~ 7条指令就会有一个分支

3. 循环级并行是指令级并行研究的重点之一

- 开发循环的不同叠代之间存在的并行性
- 循环展开（loop unrolling）技术
- 采用向量指令和向量数据表示

例如，考虑下述语句：

```
for (i=1; i<=500; i=i+1)  
    a[i]=a[i] + s;
```

1. 指令相关与流水线冲突

一个简单基本程序块

DIV.D	F2, F8, F4
ADD.D	F8, F0, F12
SUB.D	F2, F8, F14

相关是程序固有的一种属性，反映了程序中指令之间的相互依赖关系

■ 指令相关有三种类型：

- 数据相关
- 名相关（输出相关、反相关）
- 控制相关

■ 流水线冲突有三种类型：

- 结构冲突
- 数据冲突
- 控制冲突

具体的一次相关是否会导致实际冲突的发生以及该冲突会带来多长的停顿则是流水线的属性

相关与指令级并行

第 21 页

DIV.D F2, F8, F4
ADD.D F8, F0, F12
SUB.D F2, F8, F14

指令相关
流水线冲突

$$\left. \begin{array}{l} \text{指令相关} \\ \text{流水线冲突} \end{array} \right\} \text{CPI}_{\text{流水线}} \geq \text{CPI}_{\text{理想}}$$

消除相关：代码变换
避免冲突：指令调度

是否影响？
程序正确？

代码变换与指令调度必须保持的两个关键属性：

1. 数据流不变

- **定义：**数据值从其产生者指令到其消费者指令的实际流动。
- **原则：**分支指令使得数据流具有动态性，不能影响数据流，**原因：**一条指令有可能数据相关于多条先前的指令，分支指令的执行结果决定了哪条指令真正是所需数据的产生者。

2. 异常行为不变

- **定义：**保持异常行为指原来程序中是怎么发生的，改变执行顺序后还是怎么发生。
- **原则：**无论怎么改变指令的执行顺序都不能改变程序中异常的发生情况，也就是说改变指令执行顺序不能导致程序中发生新的异常属性。

■ 静态调度

- 依靠编译器对代码进行静态调度，以减少相关和冲突。
- 它不是在程序执行的过程中、而是在编译期间进行代码调度和优化。
- 通过把相关的指令拉开距离来减少可能产生的停顿。

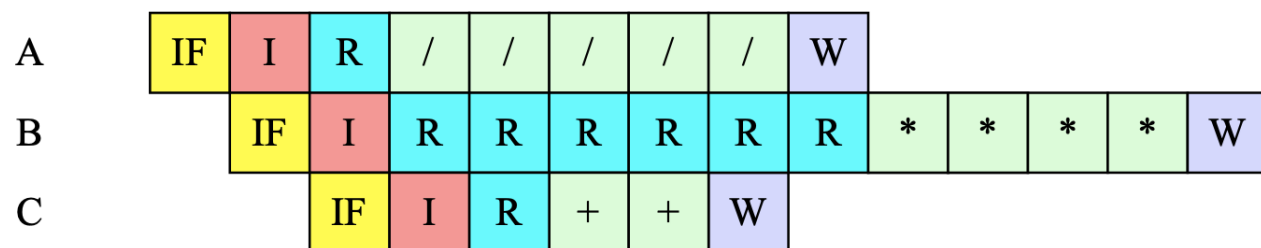
■ 动态调度

- 在程序的执行过程中，依靠专门硬件对代码进行调度，以减少数据相关导致的停顿。
- 优点：
 - ✓ 能够处理一些在编译时情况不明的相关（例如涉及到存储器访问的相关），并简化了编译器；
 - ✓ 能够使本来是面向某一流水线优化编译的代码在其它的流水线（动态调度）上也能高效地执行。
- 缺点：以硬件复杂性的显著增加为代价

动态调度的基本思想

第 23 页

结论：适当且正确地改变程序顺序不会影响程序的正确性。



A: DIV.D F0, F2, F4
B: MUL.D F6, F0, F8
C: ADD.D F10, F12, F14

乱序执行：一种将各条指令不按顺序拆散后执行的运行方式，即指令执行顺序是由输入数据可用性所决定的顺序，而不是由程序的原始数据所决定。

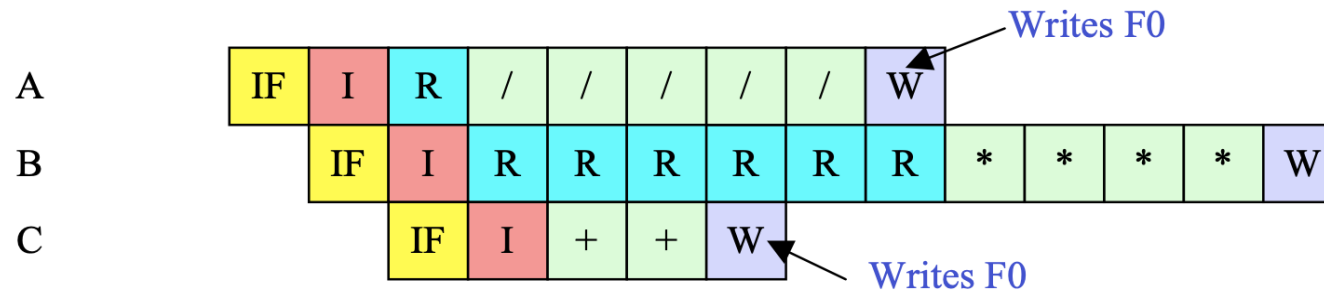
优点：可以避免因为获取下一条程序指令所引起的处理器等待，取而代之的处理下一条可以立即执行的指令。

乱序执行必须不会影响或引入新的WAR冲突和WAW冲突的

动态调度的基本思想

第 24 页

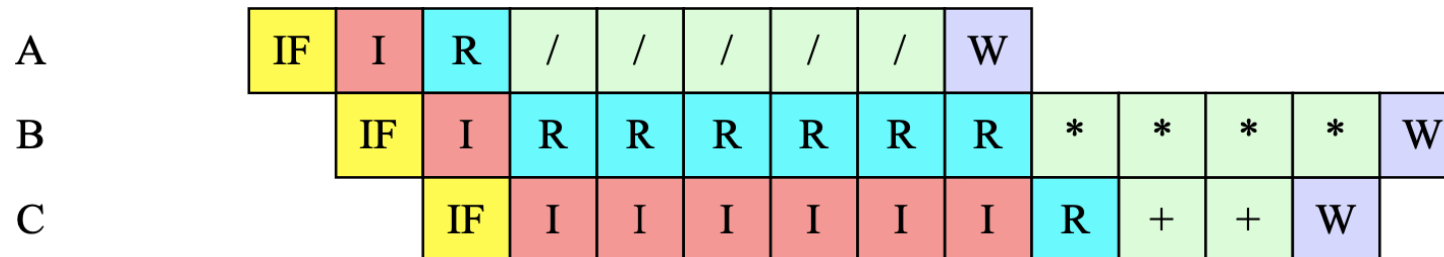
乱序执行流水线:



A: DIV.D F0, F2, F4
B: MUL.D F6, F0, F8
C: ADD.D F0, F10, F12

WAW如何处理? ← ROB技术

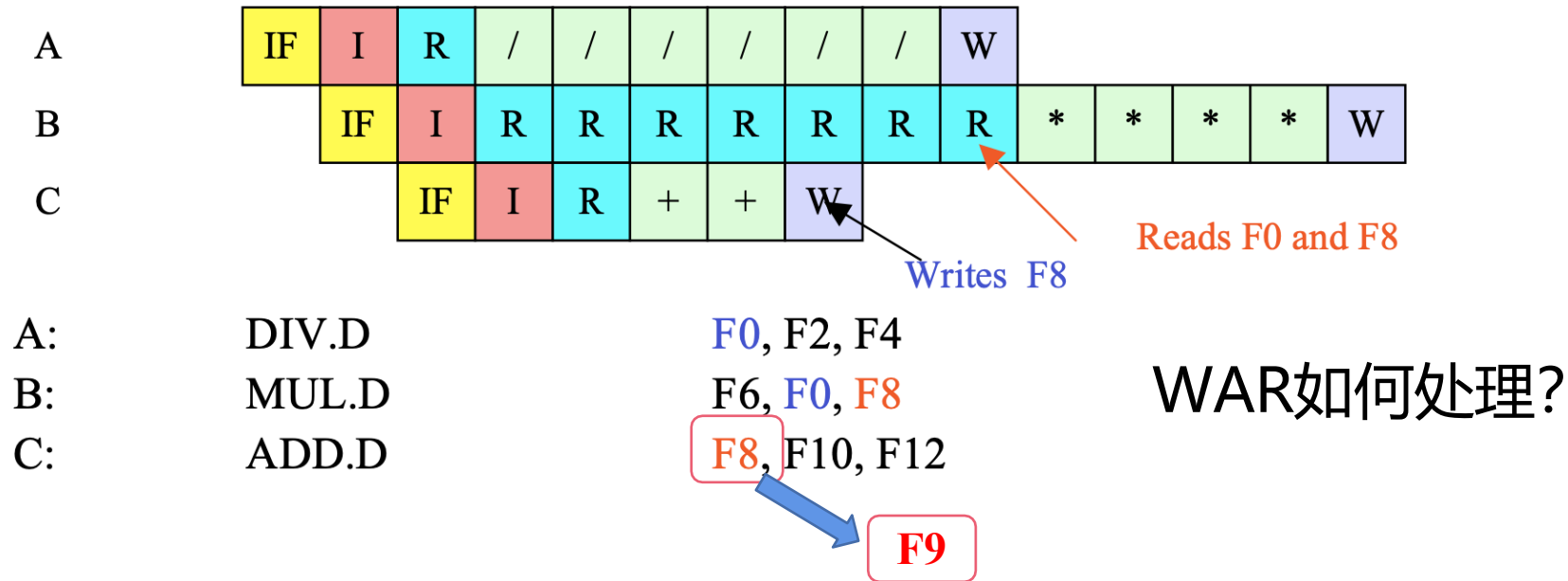
若有WAW, 则不发射



动态调度的基本思想

第 25 页

乱序执行流水线:



精确异常处理与不精确异常处理

- 精确异常：如果发生异常时处理机的现场跟严格按程序顺序执行时指令i的现场相同。
- 不精确异常：当执行指令i导致发生异常时，处理机的现场（状态）与严格按程序顺序执行时指令i的现场不同。
- 发生不精确异常的原因：因为当发生异常（设为指令i）时，流水线可能已经执行完按程序顺序是位于指令i之后的指令；流水线可能还没完成按程序顺序是指令i之前的指令。

记分牌算法和Tomasulo算法是两种比较典型的动态调度算法

基本概念

- CDC 6600 (1963) 计算机最早采用一个称为记分牌 (Scoreboard, 记分板) 的硬件实现了对指令的动态调度。
- **记分牌目标：** 在没有结构冲突时，尽可能早地执行没有数据冲突的指令，实现每个时钟周期执行一条指令
- **记分牌实现：** 硬件维护着用于记录指令的三张表
 - ✓ 执行状态表  当前流出的所有指令
 - ✓ 功能部件状态表  当前流出的所有指令是否存在结构冲突
 - ✓ 寄存器状态表  当前流出的所有指令是否存在数据冲突

显式动态寄存器重命名

第 28 页

■ 显式动态寄存器重命名

➤ 硬件维护一个映射表 (Rename Table)

- ✓ 写寄存器：为该结构寄存器 R_i (Name) 分配一个新的物理寄存器 P_j (Location) 建立映射关系；
- ✓ 读寄存器：查找映射表，查找最近一次写寄存器 R_w

Initial Mapping

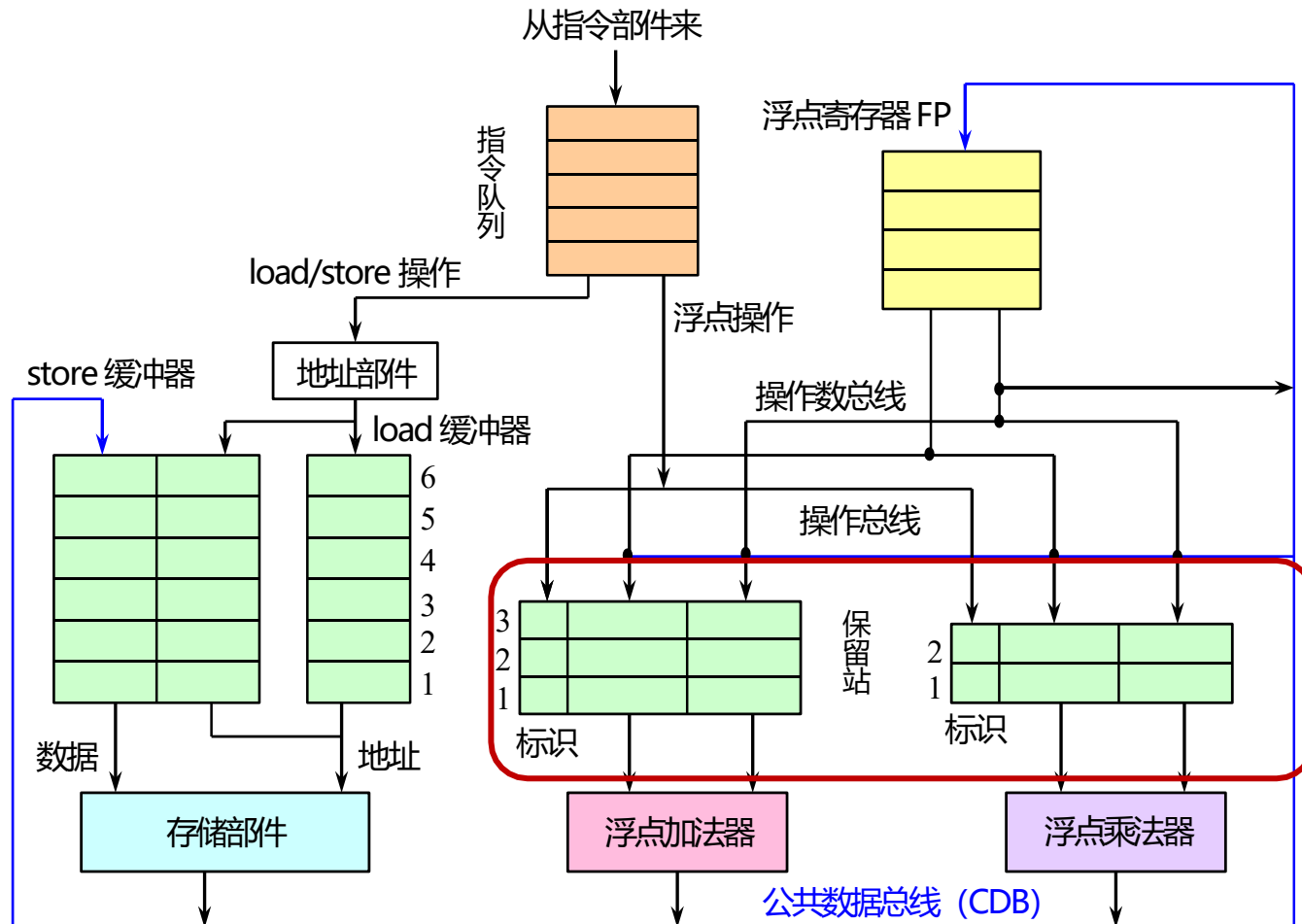
ADD R1, R2, R4
SUB R4, R1, R2
ADD R3, R1, R3
ADD R1, R3, R2

Map Table			
R1	R2	R3	R4
P1	P2	P3	P4
P5	P2	P3	P4
P5	P2	P3	P6
P5	P2	P7	P6
P8	P2	P7	P6

4个 AR
8个 PR

ADD P5, P2, P4
SUB P6, P5, P2
ADD P7, P5, P3
ADD P8, P7, P2

基于Tomasulo算法的MIPS处理器浮点部件的基本结构



保留站 (Reservation Station)

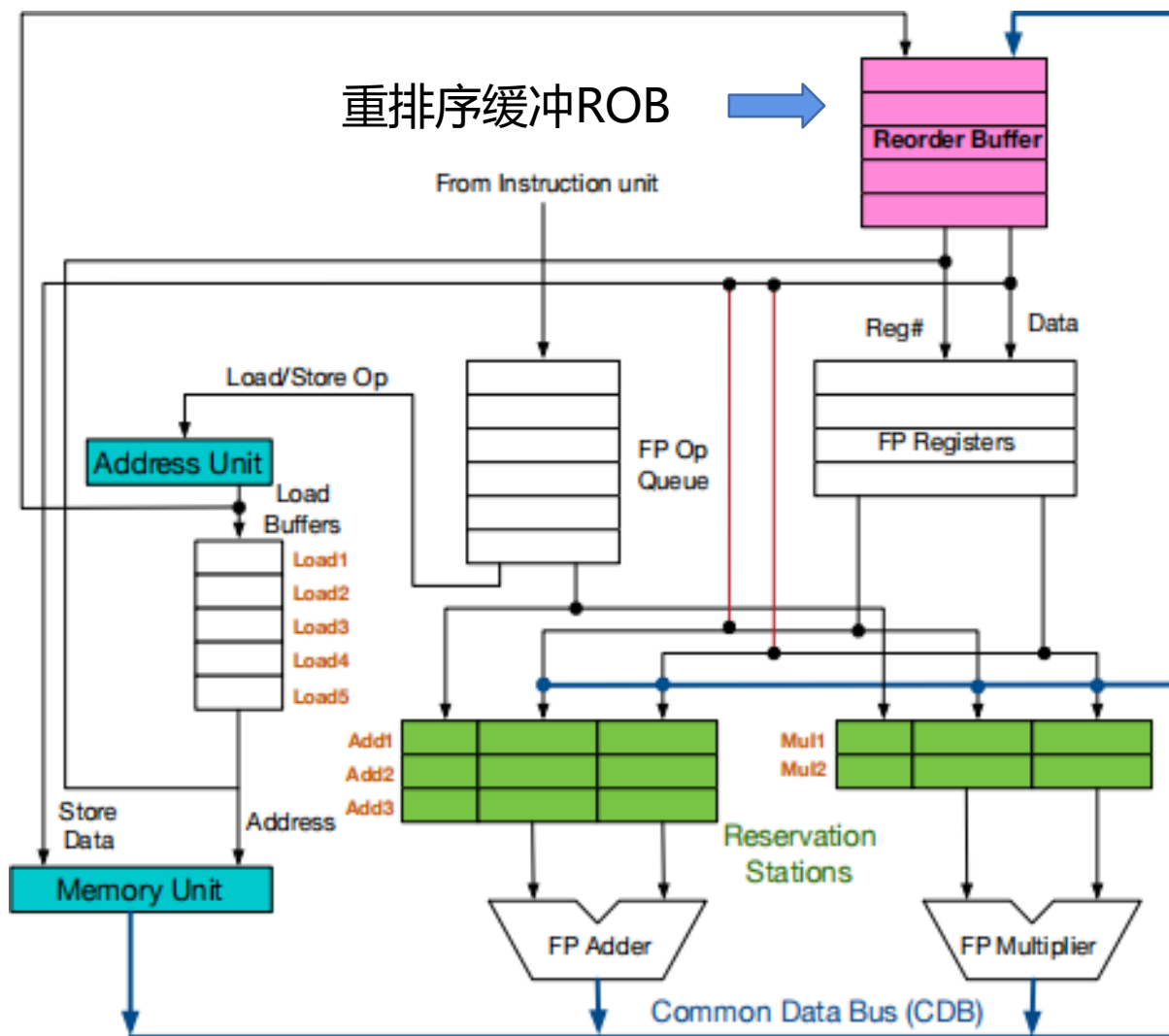
- 浮点加法器有3个保留站: ADD1-3
- 浮点乘法器有两个保留站: MULT1-2
- 每个保留站都有一个标识字段, 唯一地标识了该保留站。

公共数据总线CDB

- 所有功能部件的计算结果都是送到CDB上, 由它把这些结果直接送到 (播送到) 各个需要该结果的地方。
- 在具有多个执行部件且采用多流出 (即每个时钟周期流出多条指令) 的流水线中, 需要采用多条CDB。

重排序缓存ROB

第 30 页



改进Tomasulo算法结构图:

1. 增加了Reorder Buffer (即ROB) ;
2. CDB总线不再直通逻辑寄存器堆, 而是直通ROB;
3. 指令需要从ROB读取数据。ROB在这里就像是一个FIFO队列, 指令在发射时入队, 在提交时出队

■软件解决方案

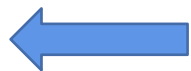
- 减少分支：循环展开 (Loop Unrolling)
- 提前形成条件码 (Reduce resolution time)
- 延迟转移 (Delay Slot) ——编译技术

■硬件解决方案

- 延迟转移 (Delay Slot) ——FLUSH技术
- 推测 (前瞻) 执行或分支预测 (Speculation - branch prediction)

■ 动态预测

➤ 转移预测缓冲器



➤ 相关转移预测器

➤ 自适应预测器

➤ 转移目标缓冲器

■ 基本原理

■ 实现

➤ 1-bit转移预测缓冲器

➤ 2-bit转移预测缓冲器

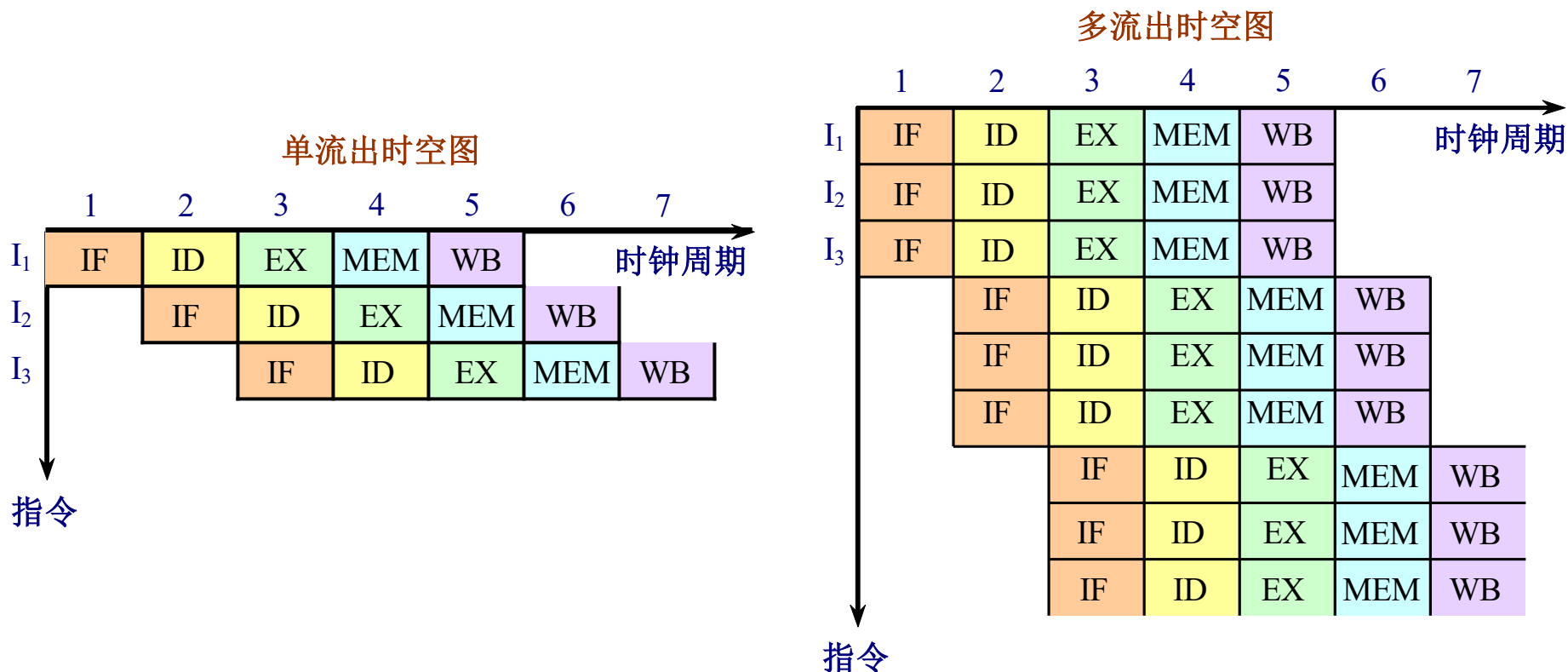
➤ n-bit转移预测缓冲器

■ 性能

多指令流出技术

第 33 页

- 在每个时钟周期内流出多条指令， $CPI < 1$ 。
- 单流出和多流出处理机执行指令的时空图对比



多流出处理机有两种基本风格：

■ 超标量 (Superscalar)

- 在每个时钟周期流出的指令条数不固定，依代码的具体情况而定。（有个上限）
- 设这个上限为 n ，就称该处理机为 n -流出。
- 可以通过编译器进行静态调度，也可以基于Tomasulo算法进行动态调度。

■ 超长指令字VLIW (Very Long Instruction Word)

- 在每个时钟周期流出的指令条数是固定的，这些指令构成一条长指令或者一个指令包。
- 指令包中，指令之间的并行性是通过指令显式地表示出来的。
- 指令调度是由编译器静态完成的。



清华大学计算机科学与技术系本科专业主修课

计算机系统结构

第6章 指令级并行及其开发—软件方法

(课程复习)

1. 指令调度：找出不相关的指令序列，让它们在流水线上重叠并行执行。
2. 制约编译器指令调度的因素
 - 程序固有的指令级并行
 - 流水线功能部件的延迟
3. 主要手段：通过循环展开、寄存器重分配和指令调度减少空转
4. 循环展开和指令调度的注意事项
 - 保证正确性；注意有效性；使用不同的寄存器；
 - 删除多余的测试指令和分支指令，并对循环结束代码和新的循环体代码进行相应的修正；注意对存储器数据的相关性分析，例如不同循环体中的存储器地址不同，则可以相互对调；注意新的相关性

1. 指令调度：找出不相关的指令序列，让它们在流水线上重叠并行执行。
2. 制约编译器指令调度的因素
 - 程序固有的指令级并行
 - 流水线功能部件的延迟
3. 主要手段：通过循环展开、寄存器重分配和指令调度减少空转
4. 循环展开和指令调度的注意事项
 - 保证正确性；注意有效性；使用不同的寄存器；
 - 删除多余的测试指令和分支指令，并对循环结束代码和新的循环体代码进行相应的修正；注意对存储器数据的相关性分析，例如不同循环体中的存储器地址不同，则可以相互对调；注意新的相关性

■ 循环展开存在问题

```
Loop:  L.D    F0, 0(R1)
        ADD.D F4, F0, F2
        S.D    F4, 0(R1)
        DADDIU R1, R1, #-8
        BNE   R1, R2, Loop
```

} 循环体是顺序结构

如果循环体是分支结构，则优化含有分支结构的循环体需要在多个基本块间移动指令，这种调度技术被称为“全局指令调度”，主要包括：

- 踪迹调度
- 超块调度

1. 基本概念：

把同时流出的或者满足特定约束的一组操作打包在一起，得到一条更长的（64位、128位或更长）的指令。



2. 与超标量的对比与分析

- 在动态调度的超标量处理器中，相关检测和指令调度基本都由硬件完成。
- 在静态调度的超标量处理器中，部分相关检测和指令调度工作交由编译器完成。
- 在VLIW处理器中，相关检测和指令调度工作全部由编译器完成，它需要更“智能”的编译器。

■ 什么是EPIC?

- 指令级并行主要由编译器负责开发， 处理器为保证代码正确执行提供必要的硬件支持， 只有在这些硬件机制的辅助下这些优化技术才能高效完成。
- 系统结构必须提供某种通信机制， 使得流水线硬件能够了解编译器 “ 安排 ” 好的指令执行顺序。

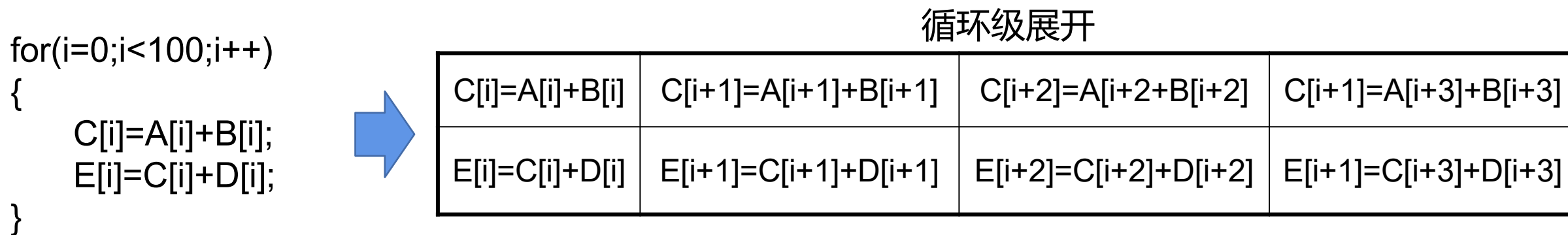
■ EPIC编译器的高级优化技术

- 非绑定分支
 - 谓词执行
 - 前瞻执行
- } 系统结构负责设计流水线硬件执行机制， 能够了解编译器 “安排” 好的指令， 动态优化执行顺序

开发更多的指令并行

第 41 页

循环级并行 (Loop-Level Parallelism)：循环中不同迭代之间的并行。由于每个循环中含有多条指令，所以它的粒度比指令级并行大。



分析：从一个循环中开发出多少并行受到的制约因素

- 迭代内部相关：同一循环迭代内部的指令相关，这属于循环体内部的基本指令调度问题；
- 结论：迭代内各语句间的数据相关对于各次迭代是否能够并行没有影响
- 条件：只要保证同一迭代内存在数据相关的各语句之间的相对顺序不变，多个循环迭代就可以并行执行。

1. 循环携带相关

- 循环携带相关是指一个循环的某个迭代中的指令与其他迭代中的指令之间的数据相关，会大大限制循环展开的效果。

例6.7 在下面的循环中，

```
for (i=1; i<=100; i=i+1) {  
    A[i+1] = A[i] + C[i];           /* S1 */  
    B[i+1] = B[i] + A[i+1];        /* S2 */  
}
```

假设数组A、B和C中所有元素的存储地址都互不相同，请问语句S1与S2之间存在哪些数据相关？

■ 复杂循环携带数据相关的处理

```
for (i=6; i<=100; i=i+1)
    Y[i] = Y[i-5] + Y[i];    // 相关距离为5
```

$Y[6] = Y[1] + Y[6]$
 $Y[7] = Y[2] + Y[7]$
 $Y[8] = Y[3] + Y[8]$
 $Y[9] = Y[4] + Y[9]$
 $Y[10] = Y[5] + Y[10]$
 $Y[11] = Y[6] + Y[11]$

```
for (i=2; i<=100; i=i+1)
    Y[i] = Y[i-1] + Y[i];    // 相关距离为1
```

$Y[2] = Y[1] + Y[2]$
 $Y[3] = Y[2] + Y[3]$

编译器必须检测出这种递归关系

- (1) 某些系统结构（特别是向量计算机）为递归提供了专门的硬件支持
- (2) 这样的递归结构中通常隐藏着大量的循环级并行

2. 存储别名分析

■ 数组是仿射的

- 如果一个一维数组 $A[m:n]$ 的下标可以被表示为形如 $a \times i + b$ 的形式，那么就称该数组是仿射的 (affine)。 i 是循环索引变量，而 m 和 n 分别表示 i 取值的下界和上界。
- 一个多维数组，如果它每一维的下标都是仿射的，那么它就是仿射的。
- 下标为 $x[y[i]]$ 是一个典型的非仿射数组。

问题：判断访问数组 $A[m:n]$ 的两条指令是否相关的问题就可以转换为判断这两条指令所访问的数组元素是否是具有相同的下标

■ GCD测试法

- 假设用下标 $a \times j + b$ 将数组A的一个元素写入存储器，而用下标 $c \times k + d$ 将数组A的一个元素从存储器中读出，这里j和k都是循环A的同一元素， $m \leq j$ ， $k \leq n$ 。显然，当且仅当 $a \times j + b = c \times k + d$ 时，这两条指令将会访问数组A的同一元素。
- 算法描述
 - ✓ 假设a、b、c、d的值均已知
 - ✓ 如果 $\text{GCD}(c, a)$ 可以整除 $(d - b)$ ，那么有可能存在存储别名。
 - ✓ 如果GCD测试的结果为假（不能整除），那么一定没有存储别名存在。

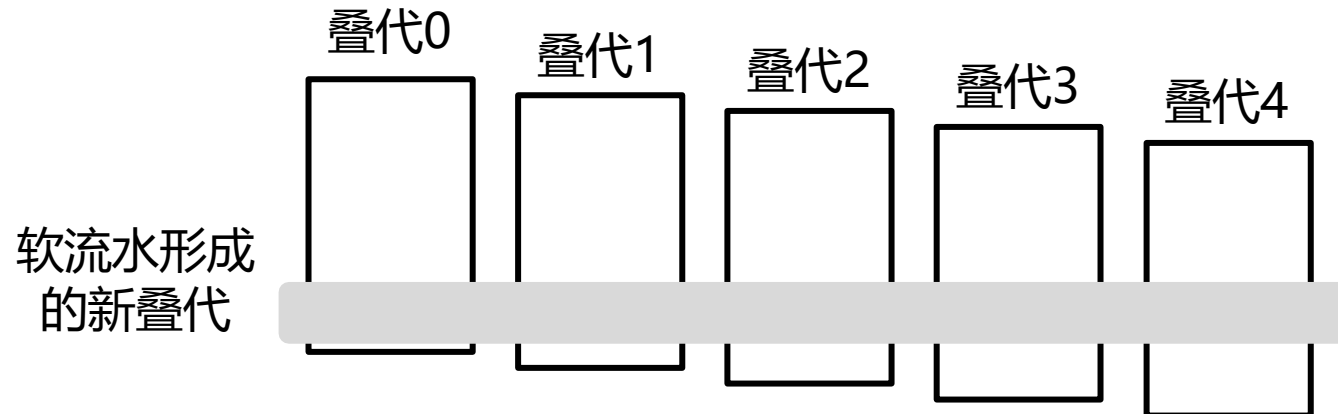
知识点：GCD(a,b)是求a和b的最大公约数，得到的最大公约数

3. 数据相关分析

- 除了检测指令之间是否存在数据相关外，编译器还会将识别出的数据相关进一步细分为**真数据相关**、**输出相关**和**反相关**等不同类型，以便利用不同的优化技术消除这些相关。
- 常用的优化有**重命名**、**值传播**、**高度消减**等。

1. 简介

- 软流水技术的核心思想是从循环不同的叠代中抽取一部分指令（循环控制指令除外）拼成一个新的循环叠代。
- 目的
 - 将同一叠代中的相关指令分布到不同的叠代中
 - 将不同叠代中的相关指令封装到同一叠代中



软流水=虚拟的硬件流水线

注意：与硬件流水线不同的是，充满或排空的指令无法被封装到任何一个新的迭代中，只能放在新循环之前或之后。

第二次作业

第 48 页

作业1：在Cray-1机器上，按照链接方式执行下述4条向量指令（//后给出了相应功能部件的执行拍数）。如果向量寄存器和功能部件之间的数据传送需要1拍，向量长度为64，则需多少拍才能得到全部结果？

$V_1 \leftarrow \text{存储器}$ //从存储器中取数，7拍

$V_2 \leftarrow V_1 + V_2$ //向量加，3拍

$V_3 \leftarrow V_2 \ll A_3$ //按 A_3 左移，4拍

$V_5 \leftarrow V_3 \wedge V_4$ //向量逻辑乘，2拍

- 注：
1. 给定的时间为对应部件从开始执行到输出第一个元素的时间
 2. 只有向量寄存器和功能部件之间传输数据需要额外1拍的时间
 3. Tstart 需要减一

第二次作业

第 49 页

作业2：某向量处理机有16个向量寄存器，其中V0~V5中分别放有向量A、B、C、D、E、F，向量长度均为8，向量各元素均为浮点数；处理部件采用两条单功能流水线，加法功能部件为2拍，乘法功能部件为3拍。采用类似Cray-1的连接技术，先计算 $(A+B) \times C$ ，在流水线不停流的情况下接着计算 $(D+E) \times F$ 。

- (1) 求此链接流水线的通过时间（设寄存器入、出各需一拍）
- (2) 假如每拍时间为50ns，完成这些计算并把结果存入相应的寄存器，此部件的实际吞吐率为多少MFLOPS？

注：由于计算指令相同，仅寄存器不同，不需要重新链接，你可以认为只做一次 $(A+B) \times C$ ，但向量长度有变化。第二问，由于需要计算MFLOPS，每个数据进行了两次浮点操作

第二次作业

第 50 页

作业3：在一台向量处理机上实现 $A=B \times S$ 计算，其中A和B是长度为 $N=200$ 的向量，S是一个标量。向量寄存器长度 $MVL=64$ ，**各功能部件从开始执行对应操作，到输出第一个结果的时间**为：取数和存数部件为12个时钟周期、乘法部件为7个时钟周期，执行标量代码的开销 $T_{loop}=15$ 个时钟周期。对一个向量元素执行一次操作的时间 T_g 是一个周期。向量寄存器和功能部件之间的数据传送需要1个时钟周期。求计算的总执行时间。

作业4：A、B两个向量存放于存储器中，其向量长度为64。设流水加法器有4级，流水线时钟周期为10ns，读出A、B向量第一对元素到流水线时，以及将数据写回存储器所需的时钟周期数2，求执行向量加法指令ADDV所需的时间。

第二次作业

第 51 页

作业5：假设有一条流水线，仅对条件转移指令使用分支目标缓冲。假设分支预测错误的开销为4个时钟周期，缓冲不命中的开销为3个时钟周期。假设：命中率为90%，预测精度为90%，分支频率为15%，没有分支的基本CPI为1。

- (1) 求程序执行的CPI。
- (2) 相对于采用固定的2个时钟周期延迟的分支处理，采用哪种方法能使程序执行速度更快？

注：

1. 开销为x个周期意味着在该指令后需要插入x个气泡
2. 固定的2个时钟周期延迟，即分支指令后面永远跟随两个气泡

第二次作业

第 52 页

作业6：假设分支目标缓冲的命中率为90%，程序中无条件转移指令的比例为5%，没有无条件转移指令的程序CPI值为1。假设分支目标缓冲中包含分支目标指令，允许无条件转移指令进入分支目标缓冲，则程序的CPI值为多少？假设原来的CPI=1.1。

提示：先用CPI=1.1把无条件转移指令的额外开销算出来。假设无条件转移指令的BTB命中率也是90%，且命中的指令不会产生额外开销，计算新的CPI。

第二次作业

第 53 页

作业7：对于两路超标量处理器，从存储器取数据有两拍附加延迟，其他操作均有一拍附加延迟，对于下列代码：

```
LW      R4, (R5)
LW      R7, (R8)
DADD    R9, R4, R7
LD      R10, (R11)
DMUL    R12, R13, R14
DSUB    R2, R3, R1
SW      R15, (R2)
DMUL    R21, R4, R7
SW      R23, (R22)
SW      R21, (R24)
```

请按以下要求进行指令调度。

(1) 假设两路功能部件中同时最多只有一路可以是访问存储器的操作，同时也最多只有一路可以是运算操作，指令顺序不变。

(2) 假设两路功能部件均可以执行任何操作，指令顺序不变。

(3) 假设指令窗口足够大，指令可以乱序流出，两路功能部件均可以执行任何操作。

第二次作业

作业7提示： 作为提示，给出(1)答案中前几行的内容

LW R4, (R5)
LW R7, (R8)
DADD R9, R4, R7
LD R10, (R11)
DMUL R12, R13, R14
DSUB R2, R3, R1
SW R15, (R2)
DMUL R21, R4, R7
SW R23, (R22)
SW R21, (R24)

LW R4, (R5)	
LW R7, (R8)	
DADD R9, R4, R7	LD R10, (R11)

- 注：
- 1. 因为一周期只能执行1条访存指令，所以第二个LW必须在第二周期才能发射
 - 2. 因为访存要附加两拍延迟，所以第5周期才能发射
 - 3. LD指令因为指令顺序要求只能在第5周期发射

第二次作业

第 55 页

作业8：设指令流水线由取指令、分析指令和执行指令三个部件构成，每个部件经过的时间为 Δt ，连续流入12条指令。分别画出标量流水处理机以及ILP均为4的超标量处理机、超长指令字处理机，并分别计算它们相对于标量流水处理机的加速比。

注：

1. 假定指令之间没有数据相关（因为执行只需要1个周期就可以完成）
2. 原本题目里面还有个超流水（实际上就是把时钟周期变为原来的 $1/\text{ILP}$ ），这个知道概念就行了

第二次作业

第 56 页

作业9：指令下面这段C代码中的所有数据相关，对于真数据相关，还应指明它们是不是循环携带相关。试分析能否从这段循环中开发出循环级并行，并说明原因。

```
for(i=2;i<100;i=i+1) {  
    a[i]=b[i]+a[i];          /*S1*/  
    c[i-1]=a[i]+d[i];        /*S2*/  
    a[i-1]=2*b[i];           /*S3*/  
    b[i+1]=2*b[i];           /*S4*/  
}
```


第二次作业

第 57 页

作业10：下面的循环中含有相关距离为1的循环携带相关，无法从中开发出大量的循环级并行。请对这段代码进行调整，使得多个循环迭代可以并行执行。

```
for(i=1;i<100;i=i+1){  
    a[i]=b[i]+c[i];        /* S1 */  
    b[i]=a[i]+d[i];        /* S2 */  
    a[i+1]=a[i]+e[i];      /* S3 */  
}
```

注：假定读写内存没有副作用，上述程序可以进行简化，从而可以挖掘循环级并行

第二次作业

作业11：假设某超标量流水线能够同时执行一条浮点指令和一条整数指令，当下面的循环程序段在该流水线上执行时，至少应被展开多少次才能消除所有的“空转”周期。假设各指令的延迟如下表所示。

```
loop: L.D      F0, 0(R1)      //取一个数组元素放入F0
      ADD.D    F4, F0, F2     //加上在F2中的标量
      S.D      F4, 0(R1)     //存结果
      DADDIU   R1, R1, #-8    //将地址减少8（每个数据占8字节）
      BNE      R1, R2, loop   //若R1不等于R，表示尚未结束；转移到loop继续
```

注：额外要求见下一页

产生结果的指令	使用结果的指令	延迟 (cycles)
浮点计算	另一个浮点计算	3
浮点计算	浮点store(S.D)	2
浮点Load(L.D)	浮点计算	1
浮点Load(L.D)	浮点store(S.D)	0

作业11要求:

1. 这题的意思是通过调度，让每个周期两个部件的至少一个在工作。
2. BNE有延迟槽需要填充。
3. 调度后的循环体内只允许出现1条DADDIU指令。
4. 数组大小是420*8byte的倍数。
5. 给出调度后的程序。

第二次作业

第 60 页

作业12: 下面这段循环完成点积运算，寄存器F2的初值为0。试结合使用循环展开和基本指令调度技术，消除其中的所有流水线“空转”周期。假设流水线延迟如下表所示，分支指令也会带来一个“空转”周期。

```
loop: L.D      F0, 0(R1)
      L.D      F4, 0(R2)
      MUL.D    F0, F0, F4
      ADD.D    F2, F0, F2
      DADDUI   R1, R1, #-8
      DADDUI   R2, R2, #-8
      BNE     R1, R3, loop
```

产生结果的指令	使用结果的指令	延迟 (cycles)
浮点计算	另一个浮点计算	3
浮点计算	浮点store(S.D)	2
浮点Load(L.D)	浮点计算	1
浮点Load(L.D)	浮点store(S.D)	0

注：下一页ppt上有要求

作业12要求:

1. BNE的延迟槽需要填充。
2. 两个数组的大小都是840*8byte的倍数。
3. 调度后的循环体内只允许出现2条DADDIU指令。
4. 需要使用下一轮循环的指令进行调度，同时保证内存足够大，访问数组下方（地址小）的空间没有副作用。只允许使用1条来自下一个循环体的load指令。